

Data platforms and cloud architecture

Data Engineering Portfolio

Selected work addressing cost, data consistency, security and capacity constraints in startup and client environments. Each case connects **measured decisions, explicit failure behavior and reusable infrastructure**, with results, rejected alternatives, ownership boundaries and work still under validation.

01	CDC / Iceberg lakehouse migration	~5B rows / 9TB · MERGE cost -97%
02	Spark on EKS / Karpenter compute economics	Allocatable disk capacity 3.2x
03	Kafka Connect CDC reliability	Up to ~355K rows recovered
04	Multiregion governance IaC and security	13 projects · 4 engineers
05	Client on-premises-to-EMR on EKS PoC	70 resources · 46% cost model

Measure before spending

Define acceptance criteria before incurring cost.

Fail closed

Never let an ambiguous state pass as success.

Verify data, not status

Check commit age and parity, not just RUNNING or lag=0.

Enable the next operator

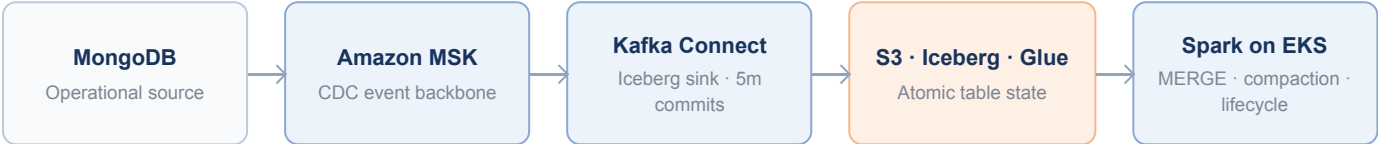
Turn individual knowledge into IaC, workflows and review rules.

1. CDC / Iceberg lakehouse migration | Wrtn Technologies

05/2026 - Present AWS S3 · EKS · MSK · Glue · Athena Technical owner

Constraints Managed-warehouse CDC apply and query costs grew with data volume. The migration had to preserve the change stream and consistency of ~200 live tables while separating physical storage, query access and lifecycle management. The platform would also take on MERGE and cleanup responsibilities previously handled by the managed engine.

Target architecture



Athena and warehouse views remain read interfaces; write-path state transitions and costs follow explicit operating rules.

Alternatives and trade-offs

Alternative	Constraint	Decision
GCS-centered design	Cross-cloud egress undermined the cost model.	Rejected; moved storage to in-region S3.
In-house sink	Weak atomic visibility of table state across writers.	Selected Apache Iceberg Kafka Connect sink.
Fixed-lookback MERGE	Runs exceeding the window could permanently miss late data.	Snapshot watermarks; explicit failure on expired history.
One restricted store + tiered views	Separate access tiers and consumer boundaries without physical copies.	Adopted as the operating standard.

Migration acceptance criteria and results

GO / NO-GO checks

- Check missing live rows and unapplied deletes separately between source and target.
- Verify snapshot-watermark continuity; fail explicitly if required history has expired.
- Test schema evolution, seeding and backfills in a development canary.

Production results

- Representative historical backfill: ~5 billion rows / 9TB.
- 196-table audit: zero missing live rows and zero unapplied deletes.
- Extended the lakehouse pattern to ingestion for eight services.

~5 billion rows · 9TB Representative service: historical backfill	196 tables · 0 / 0 Missing live rows / unapplied deletes
---	--

Ownership · Owned final architecture revisions, operating standards, the representative service cutover and technical GO / NO-GO criteria. The CTO retained business approval. A teammate owned the dbt MERGE path used as the rollback anchor; some infrastructure implementation was delegated to the team.

1. Making the lakehouse operable

Compaction · Watermarks · S3 lifecycle · Query layout

Operating invariants

Snapshot watermarks Replaced wall-clock lookback windows with incremental boundaries between the last processed snapshot and the current snapshot.	Table-shape rules Used size, change rate and delete locality to choose Copy-on-Write (CoW) or Merge-on-Read (MoR) through first-match rules.	Physical deletion Defined one deletion lifecycle: DELETE → compaction → snapshot expiry → S3 noncurrent-version expiry.
--	--	---

Encoding the CoW / MoR cost trade-off

Order	Condition	Choice and rationale
R0	Effectively append-only; changes <1%	CoW: prioritize simpler reads.
R1	Table size ≥1TB	MoR: avoid the absolute cost of full rewrites.
R2	Changes ≥10,000 rows/h AND table size ≥10GB	MoR: avoid write amplification.
R3	All remaining tables	CoW: avoid delete-file read overhead.

MERGE cost per run: -97% Large table: delete-locality repartitioning + static pruning	259TiB → 120TiB Snapshot expiry + object lifecycle: primary bucket -54%
---	---

Withdrawing an unsupported measurement

Evidence audit	Rechecked an earlier claim that row-group tuning had reduced scans from 6TB to 0.5TB against production event logs. Across 46 runs, scans still read 99.5-100% of the table; the row-group change itself saved nothing. Publicly withdrew the claim because a modeled estimate had been labeled as a measurement.
-----------------------	---

What the subsequent tests established

- Traced engine source code and found that the runtime filter never reached the row-group residual, exposing an implementation gap.
- Found that literal IN predicates enabled pruning, but lists above 200 values reverted to full scans.
- Measured a production-shaped, 131-file / 3.92GiB slice: scan volume **3,519MB → 892MB (-74.7%)**; wall time **22.7s → 6.7s**.

Scope · These are measurements from a single-bucket slice. A full-scale -99.9% figure is only an arithmetic estimate and is not presented as an achieved result.

Remaining operational work

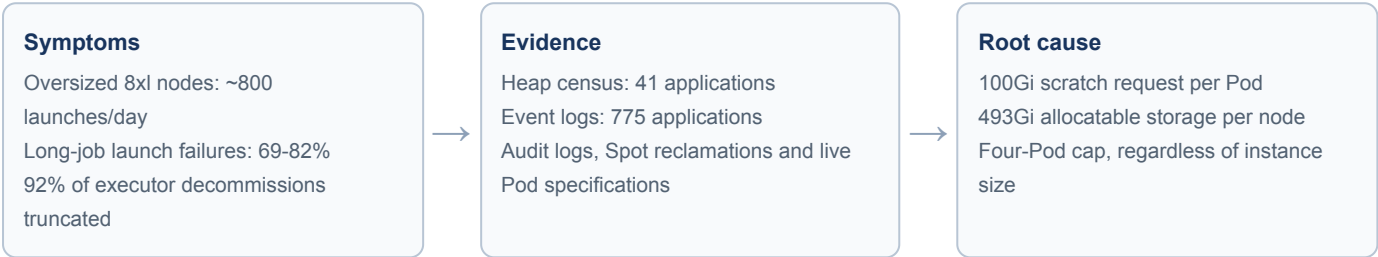
Guard scans Evaluating how to remove a target-state guard generating ~332M GET requests/day while preserving correctness.	Consumer contracts Excluded retirement of the existing bronze-shaped table from cost-saving options because it would break the consumer data contract.
---	--

2. Spark on EKS / Karpenter compute economics | Wrtn Technologies

08/2026 - Present EKS · Karpenter · EC2 Spot · EBS · NVMe Sole analyst / technical owner

Question After stopping a cost leak, the Spark executor fleet still cost ~\$830/day, and 81% of large nodes had planned memory utilization below 45%. Was Karpenter the cause, or was workload sizing preventing efficient placement?

From symptoms to root cause



Resource-by-resource decisions

Resource	Measurement	Decision
Memory	Heap p50 of 30GiB reflected delayed GC, not demand. Peak unified-memory usage <2GiB; zero spill and OOMs.	Delete-merge executor: 32g → 24g.
CPU	Changing core requests from 4.0 to 3.5 alone produced no placement gain.	Adjust memory and cores together by workload.
Ephemeral disk	Fixed 512GiB EBS volumes exposed 493GiB allocatable, capping all node sizes at four Pods.	NVMe instance-store pool + EBS fallback.
Spot capacity	Unlike the main pool, the long-job pool had no fallback tier.	Broader instance-family fallback; prevent immediate reclamation after arrival.

Acceptance criteria before spending

Alternatives rejected through measurement

- 1TiB gp3 NodeClass: added storage cost consumed half the gain.
- Exclude 8xl instances: pool weights promoted 12xl instead.
- Change pool weights: collateral impact on other workloads.
- Another dedicated pool: replaced by a simpler tiered design.

Validation of the selected design

- Proved the NVMe allocatable increase with a \$0.02 positive-control test.
- Used EBS fallback to cover capacity gaps in two Availability Zones.
- Inspected live resolved configuration to find a missing production overlay hidden by a passing development test.

493 → 1,590GiB Allocatable ephemeral storage: 3.2x; four-Pod cap removed	69-82% → 26% / 2% Long-job launch failures in observed post-fallback windows
--	--

Evidence boundary · Quantified ~\$2.1K/month in sparse-node overpayment, but do not claim realized savings before the day-seven validation gate. The long-job result is limited to relieved capacity starvation and verified fallback behavior; no causal wall-time improvement is claimed.

3. Kafka Connect CDC reliability | Wrtn Technologies

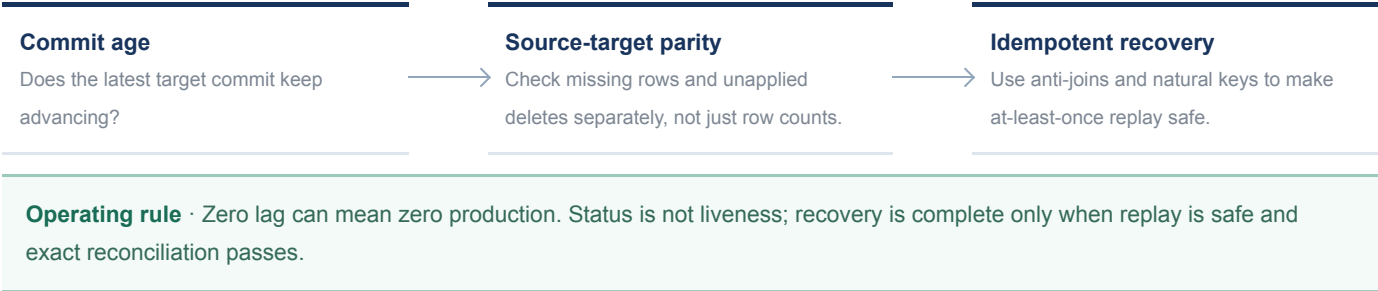
11/2025 - Present Amazon MSK · Strimzi · Kafka Connect · Iceberg Root-cause analysis owner

Problem Data movement could stop even with connector status RUNNING and consumer lag at zero. Control-plane status alone did not establish end-to-end liveness from source to target.

Three distinct silent-loss mechanisms

Failure class	Why it went undetected	Recovery and prevention
Source stall	A two-day gap across 42 tables showed lag=0; discovered five months later.	Introduced source-target parity and commit-age checks.
Zombie coordinator	11 RUNNING connectors missed ~50.7K rows over 16 minutes.	Iceberg sink runtime upgrade and anti-join recovery.
Merge-window exclusion	A downstream window turned a brief stall into permanent loss.	Bounded backfill and exact reconciliation across 12 tables, up to ~355K rows.

Defining end-to-end health



Extending the checks to connector nodes

Undrainable-node root cause

- A 30-day expiry combined with do-not-disrupt Pods caused persistent drain failure.
- Timestamps ruled out the competing explanation that the annotation arrived too late.
- Applied a development fix: set expiry to Never on the dedicated pool.

Regression prevented through peer review

- Found false positives in a reaper treating two coordinator IDs within 20 minutes as split-brain.
- Proposed a time-overlap discriminator and a leader-preservation rule.
- Blocked automation that would restart the real leader and trigger a rebalance loop.

Ownership · Handled forensic root-cause analysis, prevention and quantified backfill verification for the three incidents. Teammates operated the systems during the incidents. **The production node-pool change remains open; only the development fix is complete.**

Separate delivery evidence

Built end-to-end CDC ingestion for consent data subject to regulatory requirements. Validated **12M+ document keys** with zero duplicates and zero DLQ entries. Fixed a corruption mechanism that could remain invisible to the DLQ, rejecting the assumption that an empty DLQ proves health.

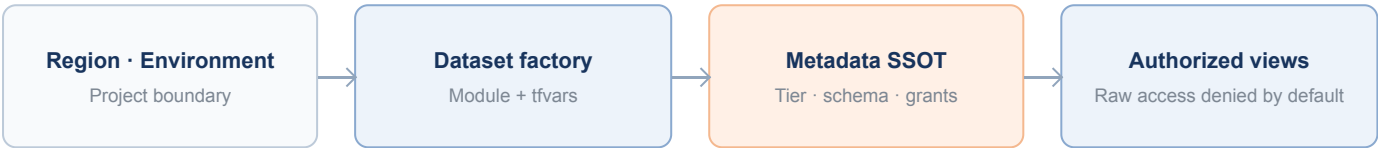
4. Multiregion governance IaC and security

| Wrtn Technologies

01/2026 - Present Terraform · GitHub Actions · BigQuery · IAM · Pub/Sub Repository creator / technical lead

Internal users IAM, authorized views and column classification were managed manually across 13 projects, ~314 datasets and ~100 users. The design needed consistent separation by region, environment and access tier, while letting the team extend it safely without depending on the original designer.

Separating infrastructure boundaries from policy



- Encoded **persona × service group × tier × region** in one Terraform source of truth.
- One time-limited PII grant entry generates a personal view, dataset authorization and decryption permissions.
- Used keyless CI/CD and plan history to detect console drift and establish IaC-only changes.

Human review as a security boundary



What was not automated

Prohibited environment-wide auto-confirmation because it could approve sensitive columns in bulk. Human review remains a security boundary.

Reducing review bottlenecks

Applied different SLAs and validated bulk-accept paths for low-risk tiers versus tiers likely to contain sensitive data.

Adoption and results

Days estimated → ~1 hour New-region deployment, including troubleshooting	7,300 + 2,100 columns Validated classification backlog cleared
---	--

- All four data engineers use the repository to change infrastructure across three regions.
- Another engineer independently extended the grant abstraction, reducing reliance on the original author.
- Replaced reviewer alerts that generated 34 mentions in 48 hours with consolidated notifications.

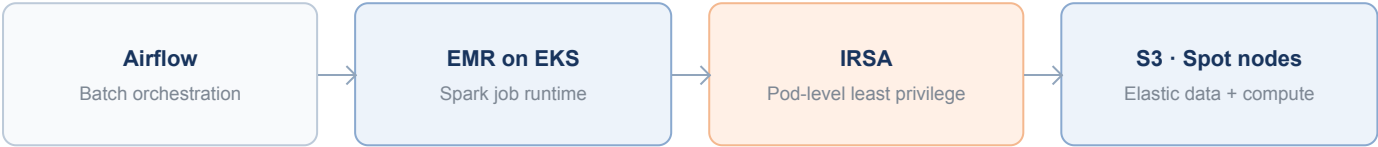
Ownership · Created the governance IaC repository and designed its dataset factory, multiregion layout, CI/CD and tier / IAM model. Incorporated a teammate's fix with authorship preserved. Final access decisions remained in the formal approval workflow.

5. Client on-premises-to-EMR on EKS PoC | Concentrix

08/2025 - 11/2025 EMR on EKS · IRSA · Spot · Terraform · Airflow Independent implementation

Client need Client-specific on-premises YARN / Spark environments required repeated console work and manual setup. The PoC tested whether growing client demand could be served through **reusable infrastructure**, **least-privilege access and workload-based cost models**, rather than a one-off migration.

PoC target architecture



Decisions based on the workload

Compute

- Chose EMR on EKS with Spot instead of an always-on cluster for batches concentrated into ~9 hours/day.
- Configured Cluster Autoscaler to scale worker nodes during execution windows.
- Separated the Airflow control-plane lifecycle from Spark workloads.

Security and reproducibility

- Connected Service Accounts, IAM roles and custom policies through IRSA for per-Pod least privilege.
- Used private EKS, a bastion and GitLab Runner to minimize external exposure.
- Defined ~70 AWS / Kubernetes resources in Terraform and verified destroy-and-reapply deployment.

Cost comparison under consistent assumptions

Scenario	Relative monthly cost	Assumptions
Existing YARN cluster	100: baseline	Same workload; reservation-discount assumptions.
EMR on EKS + Spot	~54: PoC model	Elastic compute during batch windows only.

Limitations · The ~46% reduction is a PoC projection under matched workload and discount assumptions, not a before-and-after production bill. The project ended before production adoption because there was no authority to change existing client environments and no new deployment project.

Architecture practices

Requirements

Translated batch windows, security boundaries and repeated setup costs into acceptance criteria before selecting technology.

Architecture

Combined EMR on EKS, Spot, IRSA and Terraform into a reusable operating model.

Evidence boundaries

Separated PoC feasibility from production adoption, and projected savings from realized savings.

Related materials

[Resume](#) · [English](#) · [Apache Iceberg sort-order overlap statistics proposal](#)